Source: docs\PURCHASE_JOURNEY.md | Export: 11 November 2026

# Customer purchase journey

Local implementation only. No production records, Firebase rules or indexes were changed.

## Root cause and hosted comparison

The product page still used the preparation/demo flag to disable its checkout button in production. Its dormant handler added the product to the Bag before navigating, which would have mixed Buy Now with saved Bag contents. The existing trusted order service accepted only Bag checkout. The checkout UI had no separate Summary or Payment stages. A variant-count expression rendered numeric zero before the breadcrumb for simple products. The confirmation banner also appeared before the authorized order read completed.

Public production assets inspected during this change: `/assets/index-B-p1fkEj.js`, SHA-256 `51800f62cd75fc535cb0f3be7fd07696afd485d701515f8c195f3af43316b6ba`. They contain `Checkout unavailable`, `Online checkout is unavailable` and the loyalty placeholder. They lack the new stage actions and draft namespace. Production therefore lacks this implementation. This asset comparison does not identify a deployed backend Git commit. Local starting commit was `4de81e1`.

## Routes and state

| Route | Purpose |
|---|---|
| `/`, `/shop`, `/collections/{slug}`, `/product/{slug}` | Existing real catalog navigation |
| `/login?returnTo=...`, `/register?returnTo=...` | Existing email/password authentication and validated internal return |
| `/checkout` | Delivery and item preview for the current Bag |
| `/checkout/summary` | Address, items, coupon, shipping and final total |
| `/checkout/payment` | COD selection and explicit Place COD Order |
| The three checkout routes with `?buyNow={opaqueId}` | The selected product, variant and quantity only |
| `/order-confirmed/{id}` | Thank You after an authorized server order read |
| `/order-tracking/{id}` | Authorized order status, snapshots and actual courier information |
| `/account/orders` | Customer order history and confirmation recovery |

Both sources use the same transactional review, shipping, coupon and COD engine. The source accepts only an explicit Bag discriminator or product ID, variant ID and positive integer quantity. Prices, addresses and totals cannot be supplied through the source. Server review resolves published products and enabled categories, validates owned addresses, quantities, variants and stock, and calculates integer-paise totals.

Buy Now never reads or writes Bag lines. Bag placement reads current Bag quantities and deletes those purchased lines in the same transaction as order acceptance. A concurrent Bag change changes the review fingerprint and requires another review; a receipt replay never removes later Bag additions. Inventory changes, coupon usage, immutable order snapshots and notifications retain the existing atomic acceptance policy. COD remains payment due. No payment-provider integration was added.

Drafts use account-scoped session storage with a two-hour expiry. An anonymous Buy Now source is claimed once after login or registration; it contains no address, credentials or token. Another account cannot restore the claimed draft. Selected address, coupon and confirmed summary fingerprint survive refresh. Later stages require a confirmed delivery selection; payment requires a matching current server review. Missing/expired Buy Now state never silently becomes Bag checkout. Mounting a route never places an order.

Pending placement retains the existing durable UID-scoped recovery request and operation ID, with a separate key per Buy Now flow. Recovery reads an already accepted order; retries preserve the operation identity. Price, stock, shipping, address or coupon changes during placement return an updated review and require customer confirmation again. Order confirmation and tracking fetch from the existing authenticated API and survive refresh. Unauthorized or nonexistent IDs never show the Thank You banner.

## Configuration and security

The existing Firebase Auth, Firestore and signed image infrastructure remain unchanged. Administrator access still requires both `admin: true` and an enabled `adminAccess/{uid}` record. Existing UID-owned customer profile, address and shopping rules remain intact. No new collection, Firestore rule publication, index or serverless entry point is required for this change.

Shipping must already be saved through administrator Shipping Settings. Missing or invalid settings block progression with specific owner-contact guidance; no default fee is invented by checkout. Free shipping uses the existing pre-coupon merchandise subtotal **strictly greater than** the saved threshold. At the threshold, the configured shipping fee remains payable. Coupon eligibility and limits are checked again during placement. Production configuration and hosted order placement have not been tested by this local work.

## Local acceptance

1. Run `npm.cmd run lint`, `npm.cmd test`, `npm.cmd run build`, `npm.cmd run test:admin`. The last command runs isolated local Auth/Firestore emulators and fixtures, including the complete browser journey. Never point fixture scripts at production. Optional packaging verification: `npm.cmd run test:catalog-runtime` using its documented pinned local Vercel builder.
2. In the emulator app, start at Home, open Shop and a fixture product. Verify Buy Now requires a colour for variant products and prevents quantities beyond stock. Choose a colour and quantity, sign in or register, and confirm that only that selection appears in checkout. A new customer must add an owned delivery address.
3. Select or edit an address, Continue to Summary, review the current coupon/shipping/total, Continue to Payment, and explicitly Place COD Order. Refresh each stage and use Back/Edit before placement. A direct Payment URL without the preceding state must return to Delivery.
4. Verify Thank You, the order reference, unpaid COD amount, immutable item/address summary, Track Order, View My Orders and Continue Shopping. Refresh confirmation and tracking. Confirm the saved Bag remains unchanged for Buy Now.
5. Repeat from Bag using Review checkout. Confirm only purchased quantities are removed after acceptance. Retry a simulated lost placement response and verify the same order ID and one inventory deduction.
6. Open the order as the emulator administrator, verify its notification, update fulfilment and courier fields, and refresh customer tracking. Delivered must still show payment due until an independent collection action is recorded. Another customer's order ID must remain inaccessible.

The automated suite covers shipping threshold boundaries, invalid coupons, unavailable products, ownership and administrator checks, concurrent acceptance, repeat operations and lost responses. Browser tests capture and check 320, 375, 390, 768, 1024 and 1440px product/checkout/order layouts. Screenshots and local test logs are ignored artifacts under `.cache`.

## Verification performed

- Typecheck (`npm.cmd run lint`), all 75 unit tests (`npm.cmd test`), production build (`npm.cmd run build`) and `git diff --check`: passed.
- All ten API/emulator suites: passed, including administrator/category/profile rules, catalog, appearance, shopping, delivery, shipping, orders and reporting/business regressions.
- All twelve browser regressions passed across the full-run attempts and targeted reruns: customer authentication, appearance, administrator access, categories, products, shopping, delivery, shipping, business, catalog recovery, purchase journey and COD.
- Full-run attempts stopped first at an obsolete colour-message expectation, then at browser navigation timeouts. The expectations were updated; loopback Shopping/Shipping/purchase tests now block external assets. Targeted reruns passed all remaining browser suites. The purchase width assertions wait for viewport resizing to settle. A final uninterrupted `test:admin` invocation was not rerun after those targeted passes.
- Final purchase checks include signed-out login and registration return, selected variant/quantity, preserved Bag, exact variant stock deduction, refreshed stages, Back navigation, double-click protection, invalid confirmation/selection URLs, simple-product stray-zero regression and mobile purchase/navigation separation.
- COD checks include a lost placement response with the same operation retried, restoration of a durable pending request followed by read-only recovery, one order/stock deduction, administrator fulfilment and notification, customer tracking, and collection remaining independent of Delivered status.
- Pinned Vercel builder `24.0.3` produced a `nodejs24.x` function artifact. Packaged handler initialization/JSON-status tests passed without a `tsx` loader under local Node `22.22.2`, with require(ESM) disabled and enabled. This does not verify execution in hosted Node 24.
- No production order placement, production configuration validation, commit, push, application deployment, Firebase publication or production-record mutation was performed. Public asset comparison was read-only. The real Test product was untouched.

## Changed areas

Product actions and Bag copy; checkout source validation/storage and return routes; shared delivery/order source handling; staged checkout review and COD recovery; server-backed confirmation/tracking UI; unit, API/emulator and browser regression tests. The approved customer login design and real owner-created Test product were not edited.

Files:

- UI and routes: `src/App.tsx`, `src/customer/policy.ts`, `src/products/PublicProduct.tsx`, `src/views/ProductDetailView.tsx`, `src/views/BagView.tsx`, `src/views/CheckoutView.tsx`, `src/orders/ui.tsx`.
- Shared source/review/recovery: `src/checkout/source.ts`, `src/delivery/model.ts`, `src/delivery/useCheckoutReview.ts`, `src/orders/useCodOrder.ts`, `server/delivery.ts`, `server/orders.ts`.
- Regressions and runner: `src/checkout/source.test.ts`, `scripts/orders-emulator.test.ts`, `scripts/purchase-journey-browser.test.py`, `scripts/cod-browser.test.py`, `scripts/cod-fixture-control.ts`, `scripts/product-browser.test.py`, `scripts/shopping-browser.test.py`, `scripts/delivery-browser.test.py`, `scripts/shipping-browser.test.py`, `scripts/admin-emulator-suite.ts`, `package.json`.
- Documentation: this file.